

Table of contents

Part 2 — Streaming Assimil to your iPhone with Audiobookshelf	1
Prerequisites	1
Step 1 — Inspect what’s actually on the disc	2
Step 2 — Decide which version to use	2
Step 3 — Rip the disc (on the laptop)	2
Step 4 — Copy the audio to the Pi	4
Step 5 — Run Audiobookshelf on the Pi	4
Step 6 — Initial setup in the browser	5
Step 7 — Connect the iPhone	6
Step 8 — Daily use	6
Maintenance	7
Troubleshooting	7
Recap	7

Part 2 — Streaming Assimil to your iPhone with Audiobookshelf

The payoff of this part: your Assimil *Portuguese with Ease* discs, ripped once and served from the Raspberry Pi you set up in Part 1, streaming to your iPhone from anywhere over your tailnet — with proper audiobook-style resume tracking (pause mid-sentence today, resume at the same second tomorrow).

In [Part 1](#) you built the platform: a Pi named `homelab` running Docker and on your tailnet, reachable by name from your laptop and iPhone. Now we put it to work with **Audiobookshelf** — a self-hosted media server purpose-built for spoken audio. It runs in a Docker container, scans a folder of MP3s, and serves a web UI plus a JSON API consumed by official iOS and Android apps.

Why an audiobook server and not a music server: music servers (Navidrome, Plex, Jellyfin) track “what song you played last.” Audiobook servers track “second 247 of track 14 of book X” and sync that position across every device. For Assimil — where you pause mid-sentence and resume tomorrow — that’s the whole point.

The disc lives in a CD/DVD drive on your **laptop**, so we rip there, then copy the audio to the Pi over the tailnet and run the server on the Pi.

Prerequisites

- You finished [Part 1](#): the Pi (`homelab`) is on your tailnet with Docker installed, and your laptop and iPhone are on the tailnet too.
- Your external CD/DVD drive is plugged into the laptop with the Assimil disc inserted. Verify it’s visible:

```
lsblk -o NAME,LABEL,FSTYPE,MOUNTPOINT | grep -i -E "rom|cdrom|sr"
```

You should see something like `sr0` mounted under `/media/<you>/`. If it shows but isn't mounted, replug or run `udisksctl mount -b /dev/sr0`.

Step 1 — Inspect what's actually on the disc

Don't assume the layout — Assimil's pressing varies by language and edition. List the disc root first (on the laptop):

```
ls /media/$USER/ASSBRESILMP3-3/
```

For the *Brazilian Portuguese* MP3 set you'll see three useful things:

1. **A folder ASSIMIL Brazilian Portuguese/** with 100 files named L001-LESSON.mp3 ... L100-LESSON.mp3. These are the **monolithic lessons**: one complete lesson per file, pre-assembled in playback order.
2. **Per-lesson folders** L001-Brazilian Portuguese ASSIMIL/ ... L100-..., each with ~15 small files (N1.mp3, S00-TITLE.mp3, S01.mp3 ... S07.mp3, T00-TRANSLATE.mp3, T01.mp3 ...). This is the **granular** version — every sentence and translation as its own track.
3. **The User's Manual** in HTML and PDF (mp3_Assimil_User_s_manual_*).

Plus `autorun.inf` (a Windows launcher you can ignore on Linux). The disc is a **data CD** — the fast path, no audio-CD ripping or re-encoding. A file copy is all you need.

Step 2 — Decide which version to use

Both versions contain the same audio; pick the structure based on how you'll listen.

Recommended: monolithic for daily study. One file per lesson means Audiobookshelf treats the course as a single audiobook with 100 sequential chapters. Tap “next chapter” to advance one lesson; your resume position lives at the lesson level, which is the right unit of progress for an Assimil course.

Optional: granular as a second library for drilling. When a specific sentence is giving you trouble, having S03.mp3 as its own track lets you loop it without scrubbing. Keep it around but don't make it your default — it would turn the iOS app into a wall of ~1500 unlabeled fragments.

Step 3 — Rip the disc (on the laptop)

3a. Primary library (monolithic)

```
mkdir -p ~/Audiobooks/"Assimil Brazilian Portuguese"
rsync -av --no-perms --progress \
  "/media/$USER/ASSBRESILMP3-3/ASSIMIL Brazilian Portuguese/" \
  ~/Audiobooks/"Assimil Brazilian Portuguese/"
```

`rsync` over `cp` because if the disc has a read error or you Ctrl-C halfway, re-running resumes where it stopped. `--no-perms` matters: the CD is mounted with files at `r-----` (frozen since the disc was pressed). The `-a` archive flag would copy those modes to your destination, leaving files you can barely read and directories you can't write into. `--no-perms` uses your `umask` defaults instead, so the result behaves like normal files you created. The L001 ... L100 names are already zero-padded, so Audiobookshelf orders them correctly.

3b. Secondary library (granular, optional)

```
mkdir -p ~/Audiobooks-drill/"Assimil Brazilian Portuguese (granular)"
cd /media/$USER/ASSBRESILMP3-3
rsync -av --no-perms --progress L* \
    ~/Audiobooks-drill/"Assimil Brazilian Portuguese (granular)"/
```

`--no-perms` carries even more weight here: this creates 100 *new* destination directories, and without it each would inherit the source's `r-x-----` mode, so `rsync` couldn't write the lesson's MP3s into the directory it just made. The glob `L*` matches the 100 per-lesson folders and skips the monolithic folder and the manual.

Watch the trailing slash

`rsync L*/ dest/` (with trailing slashes) copies the *contents* of each lesson folder into `dest/`, flattening 100 directories into one — same-named files (S01.mp3, T03.mp3) collide and overwrite each other. The form without the trailing slash, `rsync L* dest/`, copies each L00X folder as a subdirectory, which is what you want.

Verify after `rsync` finishes:

```
# Should print 100
ls -ld ~/Audiobooks-drill/"Assimil Brazilian Portuguese (granular)"/L* | wc
-1
```

3c. Save the manual

```
rsync -av --no-perms --progress \
    "/media/$USER/ASSBRESILMP3-3/mp3_Assimil_User_s_manual_(pdf)"/" \
    ~/Audiobooks/"Assimil Brazilian Portuguese/_manual"/
```

The leading underscore in `_manual` keeps it sorted to the bottom of the audiobook folder so Audiobookshelf scans the lessons first.

3d. Eject

```
eject /dev/sr0
```

You only do this once — from now on the files live on disk.

Step 4 — Copy the audio to the Pi

The disc is ripped on your laptop; the server runs on the Pi. Push the folders over the tailnet (this works from anywhere, but it's fastest on the same LAN):

```
rsync -avz --progress --partial --no-perms \
~/Audiobooks/ you@homelab:~/Audiobooks/
rsync -avz --progress --partial --no-perms \
~/Audiobooks-drill/ you@homelab:~/Audiobooks-drill/
```

`-z` compresses over the wire and `--partial` keeps partial files so an interrupted transfer resumes quickly — useful for a GB-scale copy. `--no-perms` means the same thing it did when ripping: don't carry the disc's read-only modes; let the Pi's umask decide.

Step 5 — Run Audiobookshelf on the Pi

SSH into the Pi (`ssh you@homelab`). Docker is already installed from Part 1. We'll run Audiobookshelf as a small **Docker Compose** project — one folder with one `compose.yaml` — the same self-contained pattern every other service in this series uses (so it's one lifecycle to learn and a single file to back up).

```
mkdir -p ~/audiobookshelf/config ~/audiobookshelf/metadata
cd ~/audiobookshelf

cat > compose.yaml <<'EOF'
services:
  audiobookshelf:
    container_name: audiobookshelf
    image: ghcr.io/advplyr/audiobookshelf:latest

    ports:
      - "13378:80"          # host 13378 -> container 80 (host 80 stays free
for Caddy)

    volumes:
      - ${HOME}/Audiobooks:/audiobooks
      - ${HOME}/Audiobooks-drill:/audiobooks-drill
      - ./config:/config
      - ./metadata:/metadata
```

```
restart: unless-stopped
EOF
```

(If you only ripped the monolithic version, drop the `${HOME}/Audiobooks-drill` line.) Then keep the runtime data out of version control and launch:

```
printf 'config/\nmetadata/\n' > .gitignore
docker compose up -d
```

What the key settings do:

- **restart: unless-stopped** auto-starts the container on boot and after crashes — exactly what you want on an always-on Pi.
- **ports: 13378:80** maps host port 13378 to the container's port 80 (we avoid host port 80 to leave it free for the reverse proxy in Part 4).
- The **volumes** expose your audio folders to the container and persist the database (listening positions, users, settings) next to the compose file, so recreating the container loses nothing. `${HOME}` is expanded by Compose to your home directory.

Verify it's running:

```
docker compose ps
curl -I http://localhost:13378      # expect HTTP/1.1 200 OK
```

Step 6 — Initial setup in the browser

From your laptop (on the tailnet), open **`http://homelab:13378`**. The first-run wizard creates the root user, then drops you at an empty dashboard.

6a. Create the root user

This is your admin account. Pick a strong password — Audiobookshelf has no password-reset flow; if you forget it, recovery means editing the SQLite database in `/config` by hand.

6b. Add the primary library

Settings (gear) → Libraries → Add Library:

- **Media type: Books.** (This changes the *data model*: Books track one resume position per book in seconds; Podcasts track per-episode played booleans. You want the first for language lessons.)
- **Library name:** Assimil Brazilian Portuguese (specific) or Audiobooks (ages better if you'll add more courses). Pick one style and stick to it.

- **Metadata providers: uncheck them all** — none will find Assimil, since it isn't a commercial audiobook, so lookups just waste scan time. (For real audiobooks later, Audible and Google Books are the useful ones.)
- **Folder:** browse to **/audiobooks** — the *container* path, not the host path. Inside Docker, `~/Audiobooks` is mounted at `/audiobooks` (from the `-v` flag in Step 5). If you see `/home/you/Audiobooks` in the picker instead, something's wrong with the volume mount.

Save. Audiobookshelf scans and creates one audiobook with 100 chapters (L001-LESSON.mp3 ... L100-LESSON.mp3) in about 30 seconds.

6c. About the drill library — add later, not now

The drill content would create 100 separate small books cluttering your app before you need them (sentence-level looping is an “active wave” activity, typically lesson 50+). You already ripped it to disk, so you can add it any time later via **Library** → **Edit** → **Add Folder** → **/audiobooks-drill** (one library) or a separate “Drills” library. No re-ripping required.

6d. Verify the scan

Open the library, open *Assimil Brazilian Portuguese*: you should see 100 numbered chapters. Click chapter 1 — it should play in the browser. If it does, the server is fully functional.

Step 7 — Connect the iPhone

1. Install **Audiobookshelf** from the App Store (by **advplyr**, same as the server). Tailscale is already installed and signed in from Part 1.
2. Open it, tap **Add Server**:
 - **Server address:** `http://homelab:13378` (or `http://100.x.y.z:13378` using the Pi's Tailscale IP).
 - **Username / password:** the root user from Step 6a.
3. Tap your library → the Assimil book → the first track. Audio should stream within a second or two — on Wi-Fi or cellular, anywhere.

Step 8 — Daily use

- **Playback speed:** 0.5×–3.0×. 0.85× is great for shadowing dialogue you don't fully understand; 1.25× for review passes. (Enable “Remember playback speed per book” in iOS settings, or it resets to 1.0× each track.)
- **Skip & scrub:** ±10s / ±30s buttons; tap-and-hold to scrub.
- **Bookmarks:** drop a named marker at the current position for phrases to revisit.
- **Cross-device progress:** pause on the iPhone, open the web UI at home, resume at the same second.
- **Offline downloads:** tap the download icon to cache a book locally — your saving grace if a network is hostile to VPNs. Downloaded audio plays with no network at all.

Maintenance

Update Audiobookshelf (on the Pi):

```
cd ~/audiobookshelf
docker compose pull && docker compose up -d
```

Because config and metadata live in `~/audiobookshelf/`, recreating the container preserves everything.

Back up listening progress with a cron job on the Pi:

```
0 3 * * * tar czf ~/backups/abs-$(date +%F).tar.gz ~/audiobookshelf/config
~/audiobookshelf/metadata
```

Add more material: drop another folder under `~/Audiobooks/` (e.g. a Pimsleur course) and click “Scan Library.” Each top-level folder becomes its own audiobook.

Troubleshooting

The iPhone app says “Cannot connect.” From the phone’s Safari, visit `http://homelab:13378`. If Safari also fails, Tailscale isn’t up — open the Tailscale app and confirm both devices show green dots. If Safari works but the app fails, you typed the wrong port (13378).

Tracks play in the wrong order. Audiobookshelf sorts by filename. Rename so they sort correctly (Lesson 01.mp3, not Lesson 1.mp3), then “Re-scan” in the web UI.

Container won’t restart after a reboot. `cd ~/audiobookshelf && docker compose ps`; if it’s down, `docker compose up -d`. If `restart: unless-stopped` isn’t taking effect, check `systemctl status docker` — Docker itself may not be enabled at boot.

rsync reports “Permission denied” on every mkdir. Check the destination mode: `ls -ld /path`. A `dr-x-----` directory is missing its write bit; fix with `chmod u+w /path` (or `chmod 755`) and re-run.

Work Wi-Fi blocks Tailscale. Tailscale falls back to DERP relays automatically (slower but works). Last resort: download lessons to the iPhone before leaving home — cached audio plays fully offline.

Recap

- **Ripped** the disc on your laptop with `rsync --no-perms`.
- **Copied** the audio to the Pi over the tailnet.
- **Ran** Audiobookshelf on the Pi as a small `docker compose` project.
- **Connected** the iPhone app to `http://homelab:13378`.

Your Assimil course is now permanently available from any device on your tailnet, with proper progress tracking and no subscription. Next, in [Part 3](#), we give the Pi a second job: blocking ads for every device on your home network with Pi-hole.